Sistem Informasi Geografis

Tugas Praktikum 6

Disusun Oleh :

M. Gymnastiar Syahputra (123140135)

Dosen Pengampu :

Muhammad Habib Algifari, S.Kom., M.TI

Alya Khairunnisa Rizkita, S.Kom., M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

Tugas 6

Link Github : https://github.com/gymnastiarsyahputra/Tugas-SIG_123140135.git

Deskripsi Tugas :

Lakukan benchmark dan optimasi query spasial pada database praktikum Anda. Dokumentasikan perbedaan performa sebelum dan sesudah optimasi.

Ketentuan Tugas:

- Buat minimal 3 spatial index pada tabel yang berbeda
- Jalankan EXPLAIN ANALYZE sebelum dan sesudah index
- Dokumentasikan speedup yang didapat (dalam%)
- Identifikasi minimal 1 query lambat dan optimasi
- Buat kesimpulan tentang pentingnya indexing

Langkah-langkah :

1. **Bagian 1: Benchmark & Pembuatan 3 Spatial Index (Sebelum dan Sesudah)**

Skenario: Kita akan mengecek relasi spasial antara transportasi.halte dan transportasi.wilayah.

a. **Langkah 1A: Hapus Index (Untuk simulasi kondisi awal)**

Untuk "mematikan" fitur pencarian cepat:

Query Query History

```
DROP INDEX IF EXISTS transportasi.idx_halte_geom;
DROP INDEX IF EXISTS transportasi.idx_wilayah_geom;
DROP INDEX IF EXISTS pertanian.idx_lahan_geom;
```

b. **Langkah 1B: Jalankan EXPLAIN ANALYZE (SEBELUM Index)**

Mencatat Execution Time yang muncul di *tab Messages*

Query Query History

```
EXPLAIN ANALYZE
SELECT h.nama, w.nama as kecamatan
FROM transportasi.halte h, transportasi.wilayah w
WHERE ST_Intersects(h.geom, w.geom);
```

Hasilnya:

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN text
1	Nested Loop (cost=0.00..2504.85 rows=1 width=29) (actual time=3.368..3.555 rows=22.00 loops=1)
2	Join Filter: st_intersects(h.geom, w.geom)
3	Rows Removed by Join Filter: 178
4	Buffers: shared hit=2
5	-> Seq Scan on halte h (cost=0.00..1.25 rows=25 width=48) (actual time=0.554..0.557 rows=25.00 loops=1)
6	Buffers: shared hit=1
7	-> Materialize (cost=0.00..1.12 rows=8 width=133) (actual time=0.002..0.003 rows=8.00 loops=25)
8	Storage: Memory Maximum Storage: 18kB
9	Buffers: shared hit=1
10	-> Seq Scan on wilayah w (cost=0.00..1.08 rows=8 width=133) (actual time=0.040..0.046 rows=8.00 loo...
11	Buffers: shared hit=1
12	Planning:
13	Buffers: shared hit=147 dirtied=2
14	Planning Time: 78.874 ms
15	Execution Time: 4.581 ms

Execution Time: 4.581 ms

c. **Langkah 1C: Buat Minimal 3 Spatial Index**

Membuat *index* GIST pada 3 tabel berbeda sesuai syarat:

Query Query History

```
CREATE INDEX idx_halte_geom ON transportasi.halte USING GIST (geom);
CREATE INDEX idx_wilayah_geom ON transportasi.wilayah USING GIST (geom);
CREATE INDEX idx_lahan_geom ON pertanian.lahan USING GIST (geom);
```

d. **Langkah 1D: Jalankan EXPLAIN ANALYZE (SESUDAH Index)**

Jalankan ulang *query* yang sama persis seperti Langkah 1B.

Query Query History

```
EXPLAIN ANALYZE
SELECT h.nama, w.nama as kecamatan
FROM transportasi.halte h, transportasi.wilayah w
WHERE ST_Intersects(h.geom, w.geom);
```

Data Output Messages Notifications

Showing rows: 1 to 13 Page

	QUERY PLAN text
1	Nested Loop (cost=0.14..110.38 rows=1 width=29) (actual time=0.113..0.148 rows=22.00 loops=1)
2	Buffers: shared hit=16 read=1
3	-> Seq Scan on wilayah w (cost=0.00..1.08 rows=8 width=133) (actual time=0.018..0.019 rows=8.00 loops=1)
4	Buffers: shared hit=1
5	-> Index Scan using idx_halte_geom on halte h (cost=0.14..13.65 rows=1 width=48) (actual time=0.013..0.015 rows=2.75 loo...
6	Index Cond: (geom && w.geom)
7	Filter: st_intersects(geom, w.geom)
8	Index Searches: 8
9	Buffers: shared hit=15 read=1
10	Planning:
11	Buffers: shared hit=51
12	Planning Time: 5.361 ms
13	Execution Time: 0.468 ms

Execution Time Turun Drastis, waktu eksekusi sekarang hanya 0.468 ms.

Kalkulasi Speedup:

- **Waktu Sebelum:** 4.581 ms
- **Waktu Sesudah:** 0.468 ms
- **Rumus:** ((Waktu Sebelum - Waktu Sesudah) / Waktu Sesudah) * 100
- **Hitungan:** ((4.581 - 0.468) / 0.468) * 100
- **Hasil Akhir:** (4.113 / 0.468) * 100 = **878.84%**

Mendapatkan **Speedup sekitar 878%** yang membuktikan bahwa *indexing* sangat esensial untuk memangkas beban kerja system.

2. Bagian 2: Optimasi Query Lambat (Anti-Pattern vs Best Practice)

Skenario: Mencari halte dalam radius 2 km dari sebuah titik koordinat.

a. **Langkah 2A: Query Lambat (Anti-Pattern menggunakan fungsi pada kolom Indexed)**

Di modul disebutkan bahwa menggunakan ST_Distance < x atau memodifikasi kolom dengan ST_Transform di dalam klausa WHERE adalah *anti-pattern* yang membuat *index* tidak bisa digunakan.

```
Query   Query History
EXPLAIN ANALYZE
SELECT nama, jenis
FROM transportasi.halte
WHERE ST_Distance(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography
) < 2000;
```

	QUERY PLAN text
1	Seq Scan on halte (cost=0.00..313.87 rows=8 width=20) (actual time=118.326..118.398 rows=6.00 loops=1)
2	Filter: (st_distance((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, true) < '2000'::double prec...
3	Rows Removed by Filter: 19
4	Buffers: shared hit=89
5	Planning:
6	Buffers: shared hit=3
7	Planning Time: 4.974 ms
8	Execution Time: 118.778 ms

Execution Time: 118.778 ms

Ini adalah query lambat (anti-pattern). Ketika kamu menggunakan fungsi ST_Distance < x di dalam WHERE, database tidak menggunakan index. Sistem dipaksa menghitung jarak matematis yang rumit antara titik kordinatmu dengan setiap halte yang ada di dalam database satu per satu secara manual. Bayangkan kalau ada jutaan data, servernya pasti langsung hang.

b. **Langkah 2B: Query Cepat (Best Practice menggunakan ST_DWithin)**

fungsi ST_DWithin sangat disarankan untuk digunakan karena fungsi ini secara internal dirancang untuk memanfaatkan *Bounding Box operator* (&&) pada spatial index.

```
Query   Query History
EXPLAIN ANALYZE
SELECT nama, jenis
FROM transportasi.halte
WHERE ST_DWithin(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography,
    2000
);
```

	QUERY PLAN text
1	Seq Scan on halte (cost=0.00..313.81 rows=1 width=20) (actual time=1.679..1.737 rows=6.00 loops=1)
2	Filter: st_dwithin((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, '2000'::double precision,...
3	Rows Removed by Filter: 19
4	Buffers: shared hit=1
5	Planning Time: 0.101 ms
6	Execution Time: 2.000 ms

Execution Time: 2.000 ms

Ini adalah *query* cepat (best practice). Dengan mengganti fungsinya menjadi ST_DWithin, waktu eksekusinya langsung terjun bebas menjadi hanya 2 milidetik. Kenapa bisa secepat ini? Karena ST_DWithin secara internal menggunakan && operator. Sistem akan membuat kotak imajiner (*bounding box*) di sekitar titik kordinatmu, lalu dengan cepat membuang semua halte yang berada di luar kotak tersebut tanpa perlu repot-repot menghitung jarak presisinya. ST_DWithin adalah cara yang BENAR untuk query radius. Selalu gunakan ini, bukan ST_Distance < x.

Kesimpulan :

Berdasarkan hasil benchmark menggunakan EXPLAIN ANALYZE, terbukti bahwa penggunaan Spatial Index (GiST) mengubah paradigma pencarian data dari pemindaian sekuensial (Seq Scan) yang memeriksa setiap baris data secara brutal , menjadi pencarian berbasis pohon hierarkis R-Tree (Bitmap Index Scan). Proses ini sangat krusial karena spatial index menerapkan Two-Phase Filter: menyaring geometri secara kasar dan super cepat menggunakan kotak pembatas (Bounding Box overlap &&), baru kemudian memverifikasi geometri aslinya.

Namun, memiliki index saja tidak cukup. Query juga harus ditulis dengan best practice. Seperti yang terlihat pada hasil pengujian, penggunaan fungsi komputasi seperti ST_Distance < x pada klausa WHERE justru memaksa database mengabaikan index. Menggantinya dengan fungsi bawaan yang index-aware seperti ST_DWithin terbukti memberikan peningkatan performa (speedup) yang sangat signifikan , sehingga meminimalisir waktu eksekusi dan beban server.